

Collector-First Redesign: Audit & Implementation Blueprint

OPTCG Price Tracker — staging prototype currently reads as a trading terminal. This report traces why, and lays out the smallest sequence of reviewable changes that gets it back to a collector product.

Branch staging **Next.js** 16.2.10 **Auth.js** next-auth ^5.0.0-beta.31 **React** 19.2.4

Prepared 2026-07-26

CONTENTS

1. Executive summary
2. Confirmed live-site failures
3. Route inventory
4. Navigation & terminology audit
5. Card-image data-flow audit
6. Pricing-model audit
7. Market Index contract
8. Collector-first page blueprint
9. Admin authorization audit
10. Recommended admin-login approach
11. Phased implementation plan
12. The first task to run
13. Commands & inspection methods
14. Open product questions

Executive summary

Every product problem in the brief traces back to four root causes, none of which require a rewrite:

1. **One line of middleware** (`apps/web/middleware.ts:11-21`) sends every signed-out visitor to `/market/movers` — a trading-terminal page — because Google OAuth has no real credentials, so *no session is ever obtainable*, so every gated route redirects, always. The `callbackUrl` it appends is never read by anything.
2. **Admin pages have no gate of their own.** Only the underlying API calls check a token; the 22 `/admin/*` pages, and the full admin nav, render for anyone who navigates there.
3. **The images are fine — the browse page isn't.** The DB, API, and a polished `CardImageFrame` component already carry images end-to-end. There is simply no card-grid route to put it on; `/search` is a text-only results list, and no `/cards` or `/sets` route exists at all.
4. **The pricing data is already well-labeled** (Yuyu-Tei sell, Yuyu-Tei buy, SNKRDUNK floor, SNKRDUNK sold are never blended) — it just has never been reduced to the single collector-facing number the brief wants: a Market Index.

None of this needs new infrastructure. It needs: a redirect fix, a real admin gate, one new page that uses a component that already exists, and one new read endpoint that reuses a query helper that already exists. The phased plan in §11 sequences exactly that, cheapest and lowest-risk first.

Confirmed live-site failures

7 of 7

GATED ROUTES REDIRECT
TO MARKET MOVERS

22

ADMIN PAGES PUBLICLY
REACHABLE

0

CARD-GRID OR SET-LIST
ROUTES EXIST

1

PLAINTEXT ADMIN TOKEN
IN LOCALSTORAGE

- **Confirmed:** `/dashboard`, `/collection`, `/wishlist`, `/grading`, `/analytics/digest`, `/analytics/buy-decisions`, `/analytics/portfolio-risk` all redirect to `/market/movers?callbackUrl=...` — reproduced directly from `middleware.ts`, not inferred.
- **Confirmed:** `/activity` is collector-personalized content that is *not* in the middleware matcher at all, so it renders unguarded for anonymous visitors — a bug of omission the brief didn't ask about but the audit surfaced.
- **Confirmed:** the full Admin (14 items) and Admin·More (8 items) nav groups render on every public page, because `SideBarNav.tsx` applies no session/role filtering and `AppHeader` is mounted individually per-page rather than gated once in a shared authenticated layout.
- **Confirmed:** images are present in the DB/API/frontend types and render correctly on `/cards/[id]` and `/collection/vault` via `CardImageFrame` — they are simply never requested by the one page acting as the public catalogue browser (`/search`).

Route inventory

ROOT CAUSE – THE MARKET MOVERS REDIRECT

Not an Auth.js callback bug, not a catch-all, not an environment condition. It is **an intentional guard that picked the wrong page, compounded by OAuth being unconfigured**:

```
// apps/web/middleware.ts:11-21
export default auth((req) => {
  if (!req.auth) {
    const redirectUrl = new URL("/market/movers", req.nextUrl.origin);
    redirectUrl.searchParams.set("callbackUrl", req.nextUrl.pathname);
    return NextResponse.redirect(redirectUrl);
  }
});
export const config = {
  matcher: ["/collection/:path*", "/grading/:path*", "/wishlist/:path*", "/das
```

Three compounding facts, each independently confirmed in the code:

1. `apps/web/src/lib/auth.ts:15-16` configures only `providers: [Google]`, and no real Google credentials exist in this environment — so `req.auth` is `null` for every visitor, always. Every gated route redirects unconditionally, not just for anonymous users.
2. `apps/web/src/app/page.tsx` is an unconditional `redirect("/dashboard")` — so even the homepage bounces straight into the guard, landing on `/market/movers`, a page branded around trading-terminal movers data, chosen as the escape hatch specifically to avoid a redirect loop.
3. `callbackUrl` is dead on arrival: `market/movers/page.tsx` never reads it, and the app's only `signIn()` call (`TopBar.tsx:92`) hardcodes `callbackUrl: "/dashboard"` regardless of where the user came from.

`/admin/*` is absent from the middleware matcher entirely — it has zero edge-level protection, separate from the redirect issue above.

ROUTE	NAV LABEL	INTENT	RENDERS	REDIRECTS TO	BACK
/	—	public	yes (immediate redirect)	/dashboard	n/a
/dashboard	Dashboard	collector- auth	blocked pre-auth	market/movers	api/ exist
/search	Cards / Search	public	yes	—	api/
/cards/[id]	(link-only)	public	yes	—	GET
/collection, /collection/vault	Collection	collector- auth	blocked pre-auth	market/movers	api/
/wishlist	Wishlist	collector- auth	blocked pre-auth	market/movers	api/
/grading	Grading	collector- auth	blocked pre-auth	market/movers	only api/ foun grad
/activity	Activity	collector- auth, but not in matcher	yes, unguarded	none (bug)	uncl
/market/movers	Market → Movers	currently double- duty as the forced anon	yes	—	api/

landing page						
<code>/market/opportunities</code>	Opportunities	trading-terminal	yes	—		api,
<code>/market/report</code>	Report	unclear	yes	—		api,
<code>/market/signals</code> , <code>/market/signal-events</code>	Signals	trading-terminal	yes	—		both
<code>/analytics/digest</code> , <code>/buy-decisions</code> , <code>/sell-decisions</code> , <code>/portfolio-risk</code>	Analytics	collector-auth, trading-terminal naming	blocked pre-auth	market/movers		all e
<code>/analytics/collection</code> , <code>/wishlist</code> , <code>/grading</code>	—	collector-auth	blocked pre-auth	market/movers		all e
<code>/admin/*</code> (22 routes)	Admin, Admin-More	admin-only	yes, publicly	none		all e

No true `/cards` list route or `/sets` route exists anywhere in `apps/web/src/app/` — both are net-new builds, not relabels, contrary to what the brief's target nav implies.

Navigation & terminology audit

Where the current IA lives

- **Sidebar config** — `SidebarNav.tsx:28-102`, four static arrays (`COLLECTOR_ITEMS`, `ANALYTICS_ITEMS`, `ADMIN_ITEMS`, `ADMIN_MORE_ITEMS`) rendered with zero auth/role gating.
- **Shell mount point** — `AppShell.tsx` (re-exported as `AppHeader`) is imported individually in all 42 page files rather than mounted once in a shared authenticated layout — which is exactly why the unfiltered admin nav leaks onto every public page.
- **Command palette** — `commandRegistry.ts`, 30 commands with a `requires_admin` flag that is **advisory only** in the data model; no enforcement filtering the rendered list by session was found, only a cosmetic `badge:"admin"`.
- **Admin gate** — `AdminAuthGate.tsx`, a client component each of the 22 admin pages renders individually when a token is missing. It gates the *data fetch*, never the page shell, chrome, or nav.

Findings

- **No dead links, no stub/TODO pages** — the palette's routes were manually reconciled against the App Router tree and it held up under inspection.
- **Palette/sidebar drift:** the sidebar links `/market/report` and `/market/movers`; the palette has commands for opportunities/signals/signal-events but not those two — the two navigation surfaces don't cover the same route set.
- **10 of 22 admin nav items are pure infrastructure concepts** with no product meaning to a collector — Job Locks, Cache, File Jobs, Release Status, Backup, Logs, Performance, Data Retention, Refresh Runs, Workflow Runs — and today they're one click away from the public sidebar.
- **Trading-terminal vocabulary:** Market Movers, Opportunities, Signals, Signal Events, Buy Decisions, Sell Decisions, Portfolio Risk — 7 of roughly 20 collector-facing nav labels, versus the brief's target vocabulary (Market Index, My Collection, Wishlist, Grading).

- **No API leakage from public pages** — none of `/search`, `/cards/[id]`, `/market/movers` call any `/api/admin/*` route. The exposure is route/nav visibility, not data.
- `/api/admin/*` **proxy routes add no security of their own** — each one (e.g. `api/admin/system-check/route.ts:10-13`) reads `X-Admin-Token` straight off the incoming browser request and forwards it verbatim. The backend token check is the only real gate today.

Proposed reduced navigation (per brief, validated against the inventory above)

TIER	ITEMS	STATUS
Public	Discover, Cards, Sets, Market Index	Discover = repaired <code>/</code> ; Cards and Sets are new builds ; Market Index = repurposed <code>/market/movers</code> + new backend contract (\$7)
Collector (authenticated)	My Collection, Wishlist, Grading	routes exist, only the redirect target needs fixing
Admin	absent unless <code>role=admin</code> ; all 22 pages grouped under <code>/admin</code>	routes exist; needs the gate built (\$9)

Everything currently under Market/Analytics that isn't Market Index itself (Opportunities, Signals, Signal Events, Buy/Sell Decisions, Portfolio Risk, Report) either folds into Collection/Wishlist as secondary detail or moves under Admin — flagged as an open question in §14, since "move to Admin" vs "remove" is a product call, not an engineering one.

Card-image data-flow audit

THE PIPELINE ALREADY WORKS

DB → API → frontend type → component is intact with no drop point. The actual gap is that the one page acting as the public catalogue browser never requests or renders an image at all.

- **Canonical field:** `Card.image_url` (`models/card.py:26`, `String(1024)`, nullable) is the single, authoritative image field.
`SnkrdunkCandidate.image_url` is a separate, scraped-listing image on unmatched candidates — no code path copies it onto `Card` on match, so it's a plausible future source, currently unused.
- **API:** a single builder, `_to_card_out` (`api/cards.py:14-39`), backs both list and detail responses and includes `image_url` in both. Confirmed in `schemas.py:226`. No serialization loss.
- **Frontend:** types and components carry it through fine — `CardImageFrame.tsx` is a fully built, vault/slab-styled component with a graceful built-in placeholder (card code, set code, rarity badge) already used on `/cards/[id]` and `/collection/vault`. It is simply never imported by `/search`, which renders results as text/badges only, with zero `<img>` or image-component references.
- **Image hosts:** none referenced anywhere — the mock price fixtures (`yuyutei_sample.json`, `snkrdunk_sample.json`) carry price data only, no image fields. There is currently no real image URL anywhere in the repo to point `remotePatterns` at.
- **Next.js image config / CSP:** `next.config.ts` has no `images.remotePatterns` at all, because `CardImageFrame` deliberately uses a raw `<img>` (with an explicit eslint-disable comment), bypassing the Next image optimizer entirely. CSP `img-src` is already `'self' data: https:` — wide open to any HTTPS host, not scoped to Yuyu-Tei/SNKRDUNK specifically.
- **Storage:** pure hotlinking. No S3/local storage/bucket/download step exists anywhere in the image path.
- **Missing-image handling:** good, but only where `CardImageFrame` is used — it falls back to a branded placeholder, never a broken `<img>`. `/search`

doesn't have a missing-image problem so much as no image slot at all.

- **Provenance:** does not exist. No `image_source`, `image_source_url`, `image_last_verified_at`, or `image_status` column anywhere. This needs to be added net-new.

Proposed image model (design only — not implemented)

FIELD	TYPE	PURPOSE
<code>image_url</code>	string	canonical primary image — already exists
<code>image_source</code>	enum	provenance: <code>official</code> / <code>yuyutei</code> / <code>snkrdunk</code> / <code>manual</code>
<code>image_source_url</code>	string, nullable	the page the image was sourced from, for audit trail
<code>image_last_verified_at</code>	timestamp	freshness — was this URL confirmed reachable/correct recently
<code>image_status</code>	enum	<code>verified</code> / <code>unverified</code> / <code>broken</code> / <code>missing</code>
<code>alt_images</code>	JSON array, nullable	optional alternate prints/angles, same shape as primary

A branded missing-image placeholder is already effectively solved by `CardImageFrame`'s fallback rendering — it just needs to be reused everywhere, not rebuilt.

Pricing-model audit

PRICES ARE ALREADY WELL-LABELED

No code anywhere blends sell + buy + floor + sold into one blurred number. The UI already shows each as a distinct, source-labeled line. The Market Index is additive, not a fix for mixed data.

- **Exactly 4 price_type values exist**, produced by the two mock adapters: Yuyu-Tei `sell` / `buy` (`mock_yuyutei.py:44-58`), SNKRDUNK `floor` / `sold` (`mock_snkrdunk.py:44-58` — one row per historical sale, each with its own real `observed_at`).
- **Storage:** one table, `price_observations` (`models/price_observation.py:9-51`) — `card_id`, `source_id`, `observed_at`, `price_type`, `price_jpy` (integer, implicitly JPY, no currency column), `condition_label`, `stock_status`, `listing_count`, raw-snapshot linkage. Full history, no aggregation, exposed as-is via `GET /cards/{id}/prices`.
- **Reusable primitive:** `latest_prices.get_latest_price_map()` (`services/latest_prices.py:69-99`) — a window-function query returning, per card, the latest observation for each `(source, price_type)` pair in one query. This is exactly the shape a Market Index calculator needs.
- **What the UI treats as "the price" today:** nothing single — `CardPricePanel` / `PriceCell` / `PriceBasisLabel` always render a value with its source + type attached (e.g. "Yuyu-Tei sell"), and missing values render literally as "not available," never blank or zero. Card detail hard-codes exactly four tiles side by side, never combined.
- **Where mixing actually happens — priority, not averaging —** `portfolio_valuation.py:169-177` prefers SNKRDUNK floor, falling back to Yuyu-Tei sell, when computing collection value; `market.py:15-21` documents an equivalent priority order for movers. This is a *different* semantic scheme from the Market Index's median-of-sources — the two will disagree on the same card unless reconciled. See §14.
- **Staleness:** client-only today — `PriceCell.tsx:4` hard-codes a 48-hour "stale" badge threshold. Nothing server-side flags staleness on the price response itself.

- **Median support:** no existing sold-price median exists, but `statistics.median()` is already an established pattern elsewhere in this codebase (median cost basis, median grade, median wishlist target) — adding one for sold prices follows convention rather than inventing one.
- **Sold vs. floor distinction:** cleanly enforced at the schema level via `price_type` alone — no ambiguity, no overlap.
- **No persisted derived-value table exists** anywhere (no `valuations` table, no continuously-updated per-card number) — everything today is computed on read from `price_observations`. A Market Index needs a new computation path; whether it also needs a new table depends on whether the trend chart in §8 requires persisted history (flagged in §14).

Pages affected by introducing Market Index: card detail, market movers, market opportunities/signals, collection + collection vault + valuation, wishlist + wishlist analytics, market report (incl. Telegram digest), price-source-health (admin).

Market Index contract

Extensible by design: each source registers a reference resolver; the index service iterates the registry and reduces whatever comes back. Adding Cardrush or Mercado later means registering a new resolver — the response shape (a list, not fixed per-source fields) never changes, so the frontend never needs to change either.

Reference rules

- **Yuyu-Tei reference** = current retail `sell` price, eligible only when the latest observation is `in_stock` and inside the freshness window. Out-of-stock or stale → not eligible, not included.
- **Yuyu-Tei buy** is retained and shown separately as a dealer-buy / liquidity indicator — explicitly excluded from the index calculation, never blended in.
- **SNKRDUNK reference** = median of recent `sold` observations, evidence type `transaction`, when the sample size meets a minimum threshold.
- **SNKRDUNK fallback** = latest `floor` listing when sold history is insufficient, evidence type `listing`, explicitly flagged as a fallback (not silently substituted).
- **Market Index** = median of all eligible source reference values. With exactly Yuyu-Tei + SNKRDUNK, the median of two values is their midpoint.
- **One eligible source** → return that value as-is, `source_count: 1`, `coverage_status: "limited"`. Never present a single value as a two-source consensus.
- **Zero eligible sources** → no numeric index; UI shows "Index unavailable."

Response shape

```
{
  "card_id": 11,
  "index_version": 1,
  "index_value_jpy": 1350,
  "calculation_method": "median_of_sources", // "median_of_sources" | "single_source"
  "source_count": 2,
  "coverage_status": "full", // "full" | "limited" | "none"
  "confidence": "high", // "high" | "medium" | "low"
  "source_values": [
```

```

{
  "source": "yuyutei",
  "reference_type": "retail_sell",
  "evidence_type": "listing",
  "value_jpy": 1200,
  "observed_at": "2026-07-26T09:30:00Z",
  "stale": false,
  "eligible": true,
  "fallback_used": false
},
{
  "source": "snkrdunk",
  "reference_type": "sold_median",
  "evidence_type": "transaction",
  "value_jpy": 1500,
  "observed_at": "2026-07-25T00:00:00Z",
  "sample_size": 2,
  "stale": false,
  "eligible": true,
  "fallback_used": false
}
],
"auxiliary_values": [
  {
    "source": "yuyutei",
    "reference_type": "dealer_buy",
    "evidence_type": "listing",
    "value_jpy": 800,
    "note": "liquidity indicator – excluded from index"
  }
],
"freshest_observation_at": "2026-07-26T09:30:00Z",
"stalest_eligible_source_at": "2026-07-25T00:00:00Z",
"stale_sources": [],
"calculated_at": "2026-07-26T10:02:11Z"
}

```

Zero-source case returns `index_value_jpy: null`, `calculation_method: null`, `coverage_status: "none"`, empty `source_values` — the frontend renders "Index unavailable" directly off `coverage_status`, never off a null-check on the number alone.

Build vs. reuse

- **Reuse:** `price_observations` schema as-is; `latest_prices.get_latest_price_map()` for the Yuyu-Tei sell + SNKRDUNK floor lookups; the existing `statistics.median()` convention;

`PriceCell` / `PriceBasisLabel` / `CardPricePanel` components, which already support arbitrary labeled price lines and just need one more mode.

- **Build new:** a windowed query for recent `sold` observations (the existing helper only returns the single latest row per series); the index calculation service itself; a decision on whether `portfolio_valuation.py`'s existing floor→sell fallback gets reconciled with or kept separate from the new index (§14); and, only if the trend chart needs persisted history rather than derive-on-read, a new snapshot table.

Collector-first page blueprint

Grounded in what already exists, not a clean-slate redesign:

- **Discover** (new — replaces the redirect-only `/`): image-led hero/featured section, notable cards, recent Market Index movers, newly added cards, popular sets. Wishlist opportunities appear only once a session exists.
- **Cards** (new route, does not exist today): image grid by default using `CardImageFrame` as-is, optional compact list view, filters for set/rarity/character/colour/language/variant. Each tile: image, character name, card code, set, rarity, variant, Market Index, movement, source coverage, last updated.
- **Sets** (new route, does not exist today): set artwork/collage, code+name, card count, index movement, highest-indexed cards; collection completion once authenticated.
- **Card detail** (exists, needs restructuring): large image already renders via `CardImageFrame`; promote Market Index to the primary value with a trend indicator, keep the existing four price tiles as the source breakdown underneath, add freshness/source-count indicators from the new contract, keep add-to-collection/wishlist actions and related-cards.
- **My Collection** (exists, needs restyling): visual binder/grid, owned quantity, condition, purchase price, current Market Index, gain/loss demoted to secondary, completion highlights.
- **Wishlist** (exists, needs restyling): image grid, target index, current index, distance to target, source freshness.

Visual language: artwork dominant, fewer dense tables, restrained analytics, strong set/rarity identity, no crypto-dashboard aesthetic, keyboard-accessible, responsive.

Admin authorization audit

THE PAGES THEMSELVES ARE PUBLIC TODAY

`/admin/*` is absent from `middleware.ts`'s matcher, and no `apps/web/src/app/admin/layout.tsx` exists. Every admin page renders its full shell/nav unconditionally — only the *data fetch* fails with a 401 until a token is entered client-side. Anyone who knows or guesses the URL sees the full admin interface.

1. **No admin role exists today.** `User` (`models/user.py:9-22`) has no `role` / `is_admin` column; the Auth.js JWT/session carries only `sub`, `email`, `name`, `picture`, `apiToken`.
2. **Yes, anyone can reach `/admin` pages** — confirmed above.
3. **Admin APIs remain protected regardless** — `require_admin_token` (`auth.py:14-28`) compares `X-Admin-Token` server-side and only auto-opens when unset *and* environment is development; staging/production always enforce it. Covered by parametrized tests over 18 admin paths.
4. **Where the token lives in the browser:** `window.localStorage`, key `admin_token` (`lib/api.ts:411-424`), entered via a form in `AdminAuthGate.tsx:31-38`.
5. **Leak surface:** `localStorage` is unencrypted, persists indefinitely, readable by any script on the origin (XSS) or extension with page access — meaningfully worse than an `HttpOnly` cookie. Not baked into the client bundle at build time (no `NEXT_PUBLIC_ADMIN*` found), so the only exposure path is post-entry client storage — but that's a real one for a shared, non-rotating, org-wide token.
6. **The Next.js proxy routes are pass-through, not a boundary** — all ~60 `/api/admin/*/route.ts` files read `x-admin-token` off the incoming browser request and forward it verbatim to the Railway API. The token round-trips through the browser on every admin action.
7. **Backend supports only the static shared token** — no dependency checks an authenticated user's role; `require_current_user` and `require_admin_token` are separate code paths.

8. **Minimal change set** to replace the client-token flow: add `role` to `User` + migration → carry `role` through the `Auth.js` `jwt` / `session` callbacks → add a real `admin/layout.tsx` with a server-side role check → add `/admin/:path*` to the middleware matcher as an *optimistic* redirect only → move `ADMIN_TOKEN` to server-only env access in the proxy routes, no longer sourced from the browser request → delete the `localStorage` flow and `AdminAuthGate`.

Recommended admin-login approach

	A • GOOGLE OAUTH + EMAIL ALLOWLIST	B • CREDENTIALS PROVIDER, SINGLE ADMIN	C • EXCHANGE SHARED TOKEN FOR HTTPONLY SESSION
Blocked today?	Yes — needs real Google Cloud Console credentials, explicitly unresolved and outside this task	No	No
New attack surface	Low	Medium — needs its own rate-limiting	Low — reuses existing token check
Fits existing code	<code>auth.ts</code> 's callback shape is ready, but blocked on the provider itself	Additive — one more provider in the same array, same callback plumbing	Smallest diff, but perpetuates the shared-secret model rather than moving toward identity
Fits "temporary, single admin, prototype"	Overkill until Google creds exist	Good fit	Good fit, weaker long-term direction

Recommendation: B — a Credentials provider added alongside the existing Google provider in `auth.ts`. It's additive, not a rewrite; it immediately produces a real `role: "admin"` claim in the session without waiting on Google Cloud Console (which this task cannot complete); and it directly obsoletes the `localStorage/X-Admin-Token`-from-browser pattern, since the Credentials login can hold `ADMIN_TOKEN` server-side only, in the already-existing proxy routes, and never send it to the client. Once real Google OAuth credentials exist, Option A becomes the natural successor — same session/role plumbing, just swap which provider populates `role`, and delete the Credentials provider.

Phased implementation plan

Repair redirects and remove broken navigation

1

● LOW RISK

OBJECTIVE	Stop bouncing every visitor to <code>/market/movers</code> ; make <code>/</code> a real page; add <code>/activity</code> to the protected matcher; make <code>callbackUrl</code> actually get consumed.
FILES	<code>middleware.ts</code> , <code>app/page.tsx</code> , <code>TopBar.tsx</code>
BACKEND	none
FRONTEND	redirect target changes to a neutral public page; sign-in flow passes the original path through
MIGRATIONS	none
TESTS	middleware/route test asserting each of the 7 routes no longer lands on <code>/market/movers</code> , and that <code>/activity</code> is now gated
ROLLBACK	revert commit, redeploy Vercel
ACCEPTANCE	none of the 7 routes redirect to Market Movers; <code>/activity</code> requires a session

Hide and protect admin functionality — pages

2

● MEDIUM RISK

OBJECTIVE	Admin nav renders for no one until a role exists; direct navigation to any <code>/admin/*</code> route is refused server-side, not just data-gated.
FILES	<code>SideBarNav.tsx</code> , new <code>app/admin/layout.tsx</code> , <code>middleware.ts</code> (<code>/admin/:path*</code>)
BACKEND	none yet
FRONTEND	admin nav groups removed from public render; unauthorized visitors to <code>/admin</code> redirected/403'd
MIGRATIONS	none
TESTS	nav render test (admin groups absent), route test (403/redirect on <code>/admin/system-check</code>)
ROLLBACK	revert; the token-gated data layer still works via direct API calls in the meantime
ACCEPTANCE	admin nav invisible everywhere; <code>/admin/*</code> unreachable without a session

Temporary admin login (Credentials provider)

2b

● MEDIUM RISK

OBJECTIVE	Real <code>role=admin</code> session; <code>ADMIN_TOKEN</code> moves fully server-side; retire the <code>localStorage</code> flow.
FILES	<code>User</code> model + migration, <code>auth.ts</code> , new <code>admin/login/page.tsx</code> , all ~60 <code>api/admin/*/route.ts</code> (mechanical edit), delete <code>AdminAuthGate.tsx</code> and the <code>localStorage</code> helpers
BACKEND	new <code>role</code> column, default <code>collector</code>
FRONTEND	broad but mechanical proxy-route edits + one new login page
MIGRATIONS	yes — add + backfill <code>role</code>
TESTS	non-admin session gets 403 on admin pages/proxies; <code>ADMIN_TOKEN</code> never appears in any client response
ROLLBACK	revert branch; token still valid for direct curl access as a fallback
ACCEPTANCE	admin login works on staging; token never reaches the browser

Establish canonical image delivery

3

● LOW RISK

OBJECTIVE	A real <code>/cards</code> grid using the existing <code>CardImageFrame</code> , since <code>/search</code> has no image slot today.
FILES	new <code>app/cards/page.tsx</code> ; <code>next.config.ts</code> only if adopting <code>next/image</code>
BACKEND	none — reuses the existing list endpoint
FRONTEND	new page; nav's "Cards" item points here
MIGRATIONS	none (image provenance fields deferred)
TESTS	grid renders with real API data, including the missing-image placeholder path
ROLLBACK	revert
ACCEPTANCE	<code>/cards</code> is image-led and uses real data

Collector-first navigation and visual components

4

● MEDIUM RISK

OBJECTIVE	Implement the reduced nav from §4; rename trading-terminal labels; fold Buy/Sell Decisions and Portfolio Risk into Collection/Wishlist as secondary views.
FILES	<code>SidebarNav.tsx</code> , <code>commandRegistry.ts</code> , affected page headers
BACKEND	none
FRONTEND	broad relabeling/restructuring, no new logic
MIGRATIONS	none
TESTS	nav/route snapshot tests updated
ROLLBACK	revert
ACCEPTANCE	nav matches the target IA; palette and sidebar agree

Market Index backend contract

5

● LOW RISK

OBJECTIVE	Implement §7's contract as a new read-only endpoint.
FILES	new <code>services/market_index.py</code> , new route in <code>api/cards.py</code> , new response schemas, new tests
BACKEND	additive endpoint, reuses <code>latest_prices.py</code> ; new windowed sold-price query
FRONTEND	none yet
MIGRATIONS	none for v1 (compute-on-read)
TESTS	0/1/2-source cases, fallback-to-floor, staleness exclusion, against the real 12-card staging dataset
ROLLBACK	remove the route
ACCEPTANCE	values match the documented rules exactly, including the correct "Index unavailable" cards

Wire Market Index into list/detail/collection/wishlist

6

● MEDIUM RISK

OBJECTIVE	Market Index becomes the primary displayed value; existing per-source tiles become secondary detail.
FILES	<code>cards/[id]</code> , <code>cards</code> (from task 3), collection + wishlist pages, <code>PriceCell</code> / <code>CardPricePanel</code> extended with an index mode
BACKEND	possible extension of <code>portfolio_valuation.py</code> pending the \$14 reconciliation decision
MIGRATIONS	none
TESTS	component tests for limited-coverage and unavailable states
ROLLBACK	revert
ACCEPTANCE	Index shown as primary everywhere it should be; old tiles remain as detail

Seed representative staging data including images

7

● LOW RISK

OBJECTIVE	Extend fixtures with image URLs and broaden mock price coverage beyond the current 3-of-12 cards, using the same non-fabrication discipline as the prior staging-data pass.
FILES	<code>services/worker/fixtures/*.json</code> , possibly the watchlist CSV
BACKEND	none structural
MIGRATIONS	none
TESTS	re-run <code>scripts/staging_smoke_test.sh</code>
ROLLBACK	documented DELETE statements, as in the prior data pass
ACCEPTANCE	most of the 12 catalogue cards show both an image and a Market Index

Validate desktop/mobile and accessibility

8

● LOW RISK, VALIDATION ONLY

OBJECTIVE	Confirm the new pages hold up responsively and pass basic a11y checks.
FILES	none / test-only
ACCEPTANCE	no critical a11y violations; usable at common breakpoints

Design automated ingestion

9

● OUT OF SCOPE

OBJECTIVE

Named as the next phase after the above — explicitly not planned or implemented here, per the task's own constraint.

12

The single first implementation task

START HERE

Task 1 — repair redirects and remove broken navigation. It's the lowest-risk change in the whole plan (frontend-only, no auth model change, no schema, no migration), and it directly fixes the single most visible live-site failure named in the brief: every gated route currently dead-ends on a trading-terminal page regardless of who's visiting. It's also a prerequisite for cleanly validating Task 2 (admin gating) — you want the redirect logic doing one clear job before adding a second guard on top of it.

Commands & inspection methods used

Entirely read-only. No file, environment variable, database, Railway, or Vercel state was touched.

```
grep -E '"(next|next-auth|@auth/core|react|react-dom)"' apps/web/package.json
find apps/web/src/app -maxdepth 2 -type d | sort
find apps/web/src -iname "middleware.ts" -o -iname "proxy.ts"
```

Four parallel read-only research passes, each confined to a distinct file set (no overlap):

- **Routes & navigation** — all 42 `page.tsx` files under `apps/web/src/app/`, `middleware.ts`, `auth.ts`, `SidebarNav.tsx`, `AppShell.tsx`, `CommandPalette / commandRegistry.ts`, `AdminAuthGate.tsx`, `lib/api.ts`, a representative `api/admin/*` proxy route, `scripts/staging_smoke_test.sh`.
- **Card images** — `models/card.py`, `models/snkrdunk_candidate.py`, `schemas.py`, `api/cards.py`, `import_watchlist.py`, `card_catalog_import.py`, frontend card types/components incl. `CardImageFrame.tsx`, `next.config.ts`, worker fixture JSON files.
- **Pricing model** — `models/price_observation.py`, `services/latest_prices.py`, `services/portfolio_valuation.py`, `services/market.py`, mock adapters, `api/cards.py`'s prices endpoint, `PriceCell / PriceBasisLabel / CardPricePanel` components, collection/grading/wishlist analytics modules.
- **Admin authorization** — `auth.ts`, `auth.py` (backend), `models/user.py`, all `api/admin/*/route.ts` proxies, all `app/admin/*/page.tsx`, `lib/api.ts` token helpers, `test_admin_auth.py`.

Official documentation fetched directly to validate current framework behavior (not assumed from training knowledge):

- nextjs.org/docs — [proxy.js](#) (v16.2.11 docs) — confirmed `middleware.ts` was deprecated and renamed to `proxy.ts` as of v16.0.0; the repo still uses the deprecated convention.
- authjs.dev — [protecting resources](#) and [extending the session](#) — confirmed the current `jwt / session` callback pattern for a custom `role` field, and the

explicit official warning that middleware/proxy must never be the sole authorization boundary.

Open product questions

Mercado

Its exact platform/domain is not assumed anywhere in this report, per instruction — needs confirmation before any resolver is built for it.

Market Index freshness threshold

Proposing 14 days for source eligibility, separate from the UI's existing 48-hour cosmetic "stale" badge. Should these align, or are they deliberately different concerns (eligibility vs. display)?

SNKRDUNK sold-median sample size

Proposing a minimum of 2 sold observations before trusting a median over the floor fallback. Product may want a higher bar (e.g. 3+) before calling it a transaction-based reference.

Collection valuation reconciliation

`portfolio_valuation.py`'s existing floor→sell fallback and the new median-based Market Index will disagree on the same card's value unless one is reconciled into the other, or they're deliberately kept as two distinct numbers ("liquidation estimate" vs. "index"). This needs a product decision before Task 6.

Market Index trend-chart persistence

Compute-on-read from historical observations, or a new persisted snapshot table written by each bounded refresh? Affects the migration scope of Task 5.

`/search` vs. the new `/cards` / `/sets` pages

Should the new image-led browse pages fully replace `/search`, or does search remain as a separate utility alongside them?

Temporary admin identity

For the Credentials-provider login: a single hardcoded admin email/password pair via env vars, or support for multiple named admins from the start?

Fate of Movers/Opportunities/Signals

Folded into Market Index as secondary views, moved wholesale to Admin, or removed from the product entirely? The audit surfaces the options; the call is product's.

Image sourcing policy

Zero image URLs exist in any current fixture or seed source. Are Yuyu-Tei/SNKR DUNK product images licensed for hotlink display in this product, or should canonical images come from an official Bandai source instead? This blocks Task 7 more than any engineering decision does.

Prepared 2026-07-26 · staging branch · read-only audit, no changes made